CircuGrid Brief — Project Overview

Srujana 2026 “Prompt to Product: Prompt for the Planet” · problem: Predictive Circular-Resource Grid for a Multi-Building Campus

1. The initial idea

Campus facilities teams audit buildings by hand, so faults and surges are noticed days late and nobody can say why they happened. The problem statement asked for a grid that fuses noisy sources, forecasts 6–12 hours ahead, classifies waste at the edge, survives connectivity gaps, explains anomalies, and nudges occupants. Our first concept (A) was a single control room covering all of that. We did not pick it by taste: we ran it through a generator–critic loop (below), and the winning framing became **CircuGrid Brief**: *forecast the surge, name the cause, fix it first*.

Data findings that shaped it (120 days, 8 buildings): 750,969 kWh ≈ 525.7 t CO_2; the two hostels and the food court use ~65% of electricity and ~95% of wet organic waste; rooftop solar covers 14.6%; and free-text staff notes reveal faults the meters never flagged (a leak, a burnt-out composter motor, a solar inverter fault, a meter calibration).

2. Development process

Stage	What we did
Rules first	A project constitution (`CLAUDE.md`) fixed the deadline, rubric, “every number computed from data” and “update the workbook after every task” before any code.
Data audit	Profiled all seven files; found duplicates, late arrivals, 37 building spellings, mixed units and the faults hidden in notes.
Idea selection	Separate ideator and evaluator agents with fresh context, three rounds, scores **7.2 → 8.0 → 8.3**. The judge caught a demo hook that contradicted the data and a leaky evaluation; we built from the best version and did not claim it passed its 8.5 gate.
Parallel plan	Function contracts (`docs/CONTRACTS.md`), phase branches and a PR workflow so two people could build concurrently; later the team built directly on `main` .
Build	Backend and ML first (pipeline, forecast, attribution, drill, classifier, impact, API), then a rough UI, then the final frontend adapted from a previous project’s design system and a reference landing template.
Verification	A DevTools-protocol harness clicked every control and recorded console errors; it found a real bug (duplicate element ids that wiped two landing sections). Clean-clone install and endpoint smoke test before deployment.

3. Tools used

- **Claude Code** (Claude Sonnet 5.5) as the coding agent; fresh Claude subagents as independent ideator and evaluator.
- **Python, pandas, scikit-learn, FastAPI/uvicorn**; vanilla HTML/CSS/JS; headless Chrome for testing and PDF generation; Git/GitHub; Docker.
- No runtime LLM: all intelligence in the product is classical ML and statistics; there is no API key dependency. Prompts and iterations are logged in the Prompt Workbook.

4. Implemented features

Feature	Status	Detail
Fusion and cleaning	Built	Dedupe, full-grid reindex, alias map, units, note parsing, sensor-fault flag
Forecast 6 h / 12 h	Built	Ridge per building vs two naive baselines; best in 5/8 at 6 h
Root-cause Brief	Built	10/10 episodes attributed with evidence
Outage Drill	Built	B04 6 h error 1.365 kWh vs 4.283 naive
Edge bin classifier + routing	Partly	84.96% (target 85%); not run on edge hardware
Impact and incentive baseline	Partly	Impact computed from data; incentive is a baseline ranking, no behaviour change measured

Web app: landing + workspace	Built	Overview, drawer, forecast explorer, brief filters, wired grid
Deployment	Partly	Dockerfile, Procfile, render.yaml; verified from a clean clone; hosted deploy done by the team

5. Final outcome

84.96% bin classifier hold-out accuracy	**5/8** buildings where ridge is best (6 h)	**1.365 kWh** Outage Drill error, B04	**10/10** anomalies attributed

The result is a working, demonstrable product: a landing page and a workspace served by one FastAPI process, running on the supplied dataset, with every number computed from it. In a live walkthrough, an operator can connect the five sources, see the B04 inverter fault, the B05 composter outage and the B07 alumni-meet surge named with their evidence, rehearse an outage, and read the impact with its assumptions.

> **Honest gaps.** The classifier is 84.96%, just under the 85% target. The forecast is close to persistence and loses to a naive baseline in three buildings. Manual numeric readings are cleaned but not fused. Latency is measured on a laptop, not edge hardware. The incentive loop is a baseline ranking. The optional LLM text feature in the original plan was not built. The dataset is synthetic, so results show feasibility, not field performance.

Repository: github.com/HARDIKSINGH150206/p2p · Team: Hardik Singh (product, ML, backend, frontend); data-cleaning exploration by NYN-05.